



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 1

Team No.: 14

Project Title: “Argument Passing”

Team Members

Roll Number	Student Name	Signature
2520030428	K. Rithwik	
2520030146	K. Srinivas	
2520030148	Bh. Shashwath	

1. Abstract

Argument passing is the mechanism by which an operating system delivers user-supplied command-line data to a newly created process. This project studies and implements this mechanism on Linux, tracing how a command typed at a shell prompt is tokenized into an argument vector, handed to a child process created with `fork()`, and loaded into the child's new program image through the `execvp()` system call, finally arriving at the program's `main(int argc, char *argv[])`.

A working mini-shell is built in C to read user commands, split them into `argv[]` arrays, spawn child processes, and confirm correct argument delivery, including edge cases such as multiple flags and quoted multi-word arguments. The project reinforces core Operating Systems concepts including process creation, process image replacement, and inter-process data transfer, providing a practical foundation for understanding how every command-line program on Unix-like systems actually starts up.

2. Problem Statement

Command-line programs rely on correctly receiving the arguments a user types, yet the path an argument takes from the shell prompt to a running process is rarely made explicit to students. Arguments can be miscounted, misplaced, or lost between the parent shell and the child process if `fork()`, `exec()`, and argument tokenization are not handled correctly, leading to programs that behave unpredictably and are difficult to debug. There is a need for a clear, hands-on demonstration of how the OS transfers `argc` and `argv` from user input, through process creation, into a program's `main()` function.

3. Objectives

1. Study how the operating system and C runtime deliver command-line arguments to a program through argc and argv.
2. Design and implement a mini-shell in C that reads, tokenizes, and passes user commands as an argument vector.
3. Demonstrate the fork()-exec() sequence, showing how execvp() replaces a process image while carrying the argument array forward.
4. Test and validate correct argument handling across edge cases such as empty input, multiple flags, and quoted arguments.

4. Proposed Methodology

The shell will read a raw command line from the user and tokenize it on whitespace and quotes into a NULL-terminated argv[] array. The shell process then calls fork() to create a child process, which is an exact copy of the parent. In the child, execvp(argv[0], argv) is called to replace the child's memory image with the target program while passing the same argument vector through to the new program's entry point.

The new program receives its arguments through main(int argc, char *argv[]) and executes normally. Meanwhile, the parent shell calls wait()/waitpid() to pause until the child terminates, then displays the result and prompts for the next command. This fork-exec-wait cycle will be implemented, traced with GDB, and tested against a range of sample commands to verify that arguments are received intact at every stage.

5. Operating Systems Concepts / Linux APIs Used

(List the relevant Operating Systems concepts, Linux system calls, POSIX APIs, or mechanisms that will be used.)

OS Concept / Linux API / System Call	Purpose in the Project
fork()	Creates a child process to execute the user's command independently of the shell.
execvp() (exec family)	Replaces the child process image with the target program, passing argv[] forward.
wait() / waitpid()	Suspends the parent shell until the child process finishes execution.
argc / argv (main() parameters)	Receives the tokenized argument count and vector inside the newly exec'd program.
exit() / _exit()	Terminates the child process and returns a status code to the parent shell.
strtok() / manual tokenizer	Parses the raw command-line string into individual arguments before fork().

6. Individual Contribution

(Clearly specify the responsibility of each team member in the project.)

Roll Number	Student Name	Individual Responsibility
2520030428	K. Rithwik	
2520030146	K. Srinivas	
2520030148	Bh. Shashwath	

7. Tools / Platforms / Software Used

(List the programming languages, operating system, libraries, tools, and software planned for the project.)

Tool / Platform / Software	Purpose
Linux / Ubuntu	Base operating system for development, testing, and running the shell.
C / C++	Language used to implement the mini-shell and demonstrate system calls.
GCC Compiler	Compiles the C source code into an executable shell program.
GDB Debugger	Traces process creation and verifies argument delivery at runtime.
Linux man pages / POSIX docs	Reference for fork(), exec(), and wait() system call semantics.

8. Expected Outcome

(Describe the expected functionality, output, and outcome of the project.)

The project will deliver a working mini-shell that correctly parses and passes command-line arguments to child processes, along with a clear demonstration of the fork()-exec()-wait() sequence and how argc/argv survive process image replacement.

Testing will confirm reliable argument handling across edge cases including no arguments, multiple flags, and quoted multi-word arguments, resulting in a practical reference that strengthens understanding of process creation and system-call level programming for future Operating Systems coursework.

9. GitHub Repository

GitHub Repository Link: https://github.com/ksrinivas7427-prog/868_OSSP

Repository Created: _____

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name:

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____